# Database Schema (ERD) & Development Environment

## University Café Cash App (TransactionSystem)

Suleman Ahmed · Pakistan Global Institute

October 2026

## 1. Storage model

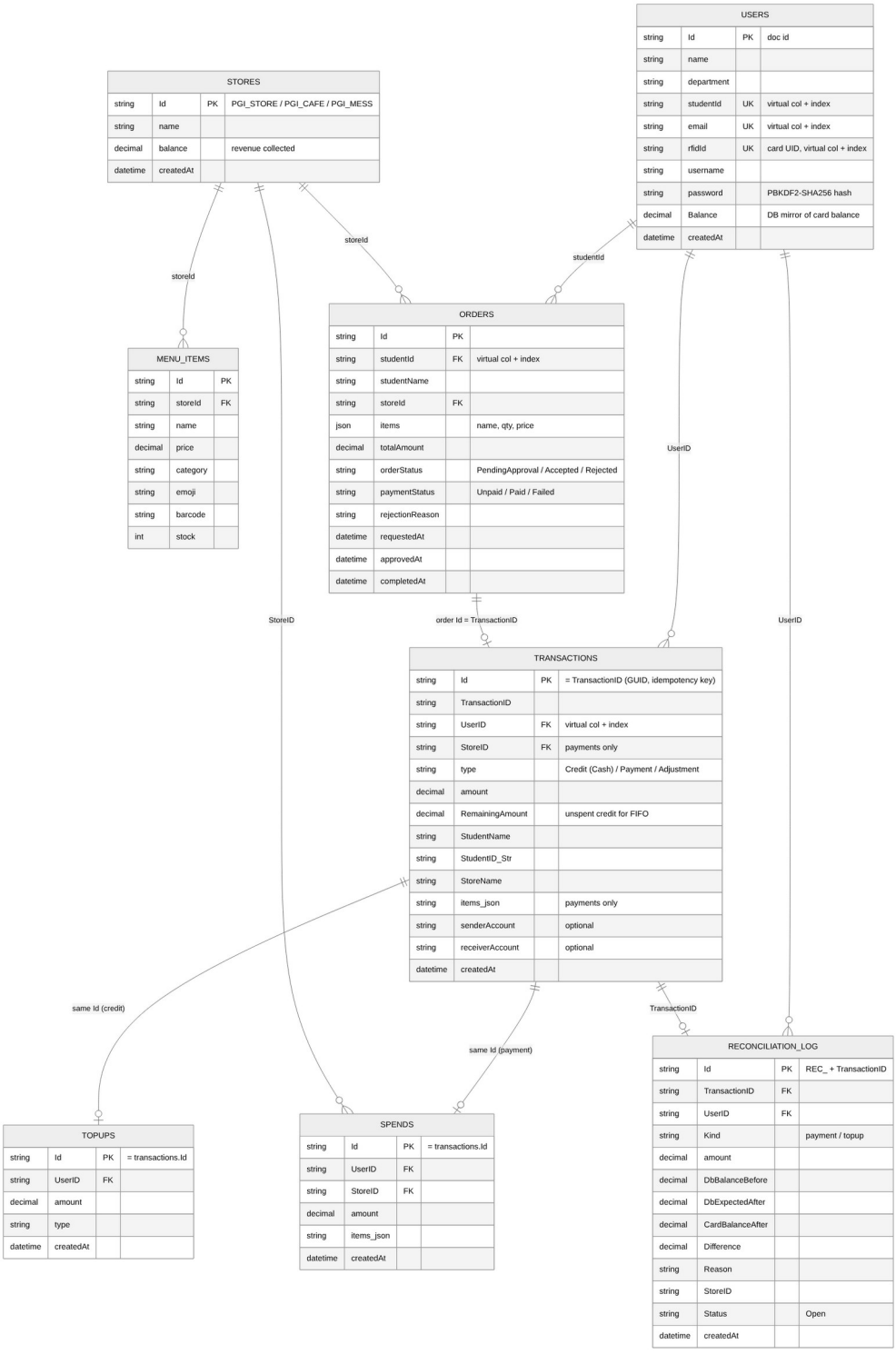
The app uses **MySQL 8** through **MySqlConnector 2.5.0** with plain ADO.NET (no ORM). Every *collection* is stored as a **document table** with two columns:

| Column | Type | Meaning |
|---|---|---|
| `Id` | `VARCHAR(255) PRIMARY KEY` | Document key (student doc ID, transaction GUID, store code …) |
| `Data` | `JSON NOT NULL` | The record itself, a JSON object |

Fields that are searched often are exposed as **indexed virtual columns** (`GENERATED ALWAYS AS (JSON_UNQUOTE(JSON_EXTRACT(Data, '$.field'))) VIRTUAL`), so lookups by card UID or student ID use an index instead of scanning JSON. The tables and indexes are created automatically by `DatabaseHelper.InitializeDatabaseAsync()` on first start; `schema.sql` in this folder contains the same DDL for manual setup.

Relationships are **logical**: they are enforced by the service layer inside one MySQL transaction per money operation (row locks with `SELECT … FOR UPDATE`), not by SQL foreign keys.

# 2. Entity-Relationship Diagram




*Entity-Relationship Diagram*

The editable source is `ERD.mmd` (Mermaid; GitHub renders it directly).

## 3. Tables and fields

### users — one row per student

| Field | Type | Notes |
|---|---|---|
| `Id` | string (PK) | Document ID |
| `name`, `department` | string | |
| `studentId` | string | Unique; indexed virtual column |
| `email` | string | Unique; indexed virtual column |
| `rfidId` | string | Card UID, unique; indexed virtual column |
| `username` | string | Same as `studentId` |
| `password` | string | Salted PBKDF2-SHA256 hash |
| `Balance` | decimal | Database copy of the balance (the card holds the live value) |
| `createdAt` | datetime | |

### transactions — every money movement

| Field | Type | Notes |
|---|---|---|
| `Id` / `TransactionID` | string (PK) | GUID created before the card write; idempotency key for retries and offline replay |
| `UserID` | string (FK → users) | Indexed virtual column |
| `StoreID`, `StoreName` | string (FK → stores) | Payments only |
| `type` | string | `Credit (Cash)`, `Payment`, `Adjustment…` |
| `amount` | decimal | |
| `RemainingAmount` | decimal | Unspent part of a credit; payments consume credits oldest-first (FIFO) |
| `StudentName`, `StudentID_Str` | string | Denormalized for reports |
| `items_json` | string | Purchased items (payments) |
| `senderAccount`, `receiverAccount` | string | Optional |
| `createdAt` | datetime | |

### topups and spends — split views of transactions

Each credit is also written to `topups`, and each payment to `spends`, **with the same `Id` and the same JSON** as its `transactions` row, inside the same database transaction. They exist so history screens can list deposits and purchases separately without filtering.

### stores — the outlets

| Field | Type | Notes |
|---|---|---|
| Id | string (PK) | PGI_STORE, PGI_CAFE, PGI_MESS |
| name | string | |
| balance | decimal | Revenue collected by the outlet |
| createdAt | datetime | |

### menu_items — catalog per outlet

Id, storeId (FK → stores), name, price, category, emoji, barcode, stock.

### orders — self-service orders (written by the external ordering app)

| Field | Type | Notes |
|---|---|---|
| Id | string (PK) | Also used as the TransactionID when the order is charged |
| studentId | string (FK → users) | Indexed virtual column |
| studentName, storeId | string | |
| items | JSON | List of {name, qty, price} |
| totalAmount | decimal | |
| orderStatus | string | PendingApproval → Accepted / Rejected; indexed virtual column |
| paymentStatus | string | Unpaid / Paid / Failed |
| rejectionReason | string | |
| requestedAt, approvedAt, completedAt | datetime | |

### reconciliation_log — card/database disagreements

Written when a card balance after an operation does not match what the database expected. Id = REC_ + TransactionID; fields TransactionID, UserID, StudentID_Str, StudentName, Kind, amount, DbBalanceBefore, DbExpectedAfter, CardBalanceAfter, Difference, Reason, StoreID, Status (Open), createdAt.

### Relationships

| From | To | Cardinality | Key |
|---|---|---|---|
| users | transactions | 1 : many | transactions.UserID |
| transactions | topups / spends | 1 : 0..1 | same Id |
| stores | spends | 1 : many | StoreID |

| From | To | Cardinality | Key |
|---|---|---|---|
| stores | menu_items | 1 : many | `storeId` |
| users | orders | 1 : many | `orders.studentId` |
| orders | transactions | 1 : 0..1 | order `Id` = `TransactionID` |
| users / transactions | reconciliation_log | 1 : many / 1 : 0..1 | `UserID` / `TransactionID` |

### Local files (not in MySQL)

| File | Purpose |
|---|---|
| `offline_queue.json` | Money operations waiting to be replayed while the DB is unreachable |
| `offline_failed.json` | Operations that failed 5 replays, set aside for the admin |
| `error.log` | Application log (`AppLog`) |
| `appsecrets.json` | Connection string, role password hashes, card master key (git-ignored) |

## 4. Development environment

### Hardware and OS

| Item | Requirement |
|---|---|
| OS | Windows 10 or 11 (WinForms; the reader DLLs are 32-bit Windows libraries) |
| RFID reader | CR100 USB MIFARE reader with `cr100usb.dll` and `UsbMiCard.dll` (included in the repo, copied to the output folder) |
| Cards | MIFARE Classic 1K |
| Printer | Optional, any Windows printer for receipts and reports |

### Software

| Tool | Version / notes |
|---|---|
| .NET SDK | .NET 10 (`TargetFramework` = `net10.0-windows`) |
| IDE | Visual Studio with the *.NET desktop development* workload, or VS Code + C# Dev Kit |
| Database | MySQL 8.0+ (local for development, hosted for production) |
| Git | For cloning and version control |
| PowerShell | For the helper scripts below |

## NuGet packages

| Package | Version | Used for |
|---|---|---|
| MySqlConnector | 2.5.0 | Database access |
| Newtonsoft.Json | 13.0.4 | JSON documents and offline queue |
| ClosedXML | 0.105.0 | Excel export |

## Project settings

- Output type: Windows application (`WinExe`), `UseWindowsForms = true`
- `PlatformTarget = x86` — **required**, because the reader DLLs are 32-bit
- Nullable reference types and implicit usings enabled

## Folder structure

```
TransactionSystem/
├── Program.cs, Form1.cs, LoginForm.cs
├── DashboardForm.cs
├── Controls/
│   ├── StudentControl, AdminControl
│   ├── CafeOrdersControl, TransactionsControl
│   └── DashboardControl, ReceiptForm, SalesReportForm
├── Services/
│   ├── TransactionService, RegistrationService
│   ├── RFIDReaderService, CardSecurity, CardWriteHelper
│   ├── OfflineSyncService, SessionService
│   ├── DashboardService, ExportService
│   └── PasswordHasher, AttemptLimiter, AppLog
├── Data/
│   ├── DatabaseConfig  (loads appsecrets.json)
│   └── DatabaseHelper
├── cr100usb.dll, UsbMiCard.dll
├── Build-Package.ps1, Setup.bat
├── Set-RolePassword.ps1, Create-SourceZip.ps1
└── appsecrets.json  (git-ignored, never commit)
```

## Setup steps

1. **Clone** the repository and open `TransactionSystem.sln`.
2. **Install MySQL 8** and create an empty database (for example `transaction_system`) and a user with rights on it. Optionally run `schema.sql`; otherwise the app creates the tables on first start.
3. **Create `appsecrets.json`** next to the project file with these keys: `ConnectionString`, `AdminPasswordHash`, `StorePasswordHash`, `CafePasswordHash`, `MessPasswordHash`, `DashboardUser`, `DashboardPasswordHash`, `CardMasterKey` (base64), `RequireCardSignature`, `CardRekeyEnabled`. The app refuses to start if the file is missing or incomplete. Keep the file out of git; it is already in `.gitignore`.

4. **Set operator passwords** with `.\Set-RolePassword.ps1 -Role Admin` (and Store, Cafe, Mess, Dashboard). It writes PBKDF2 hashes into `appsecrets.json`; passwords are never stored in plain text.
5. **Connect the CR100 reader** by USB. The two DLLs are copied to the output folder automatically.
6. **Build and run** in Visual Studio with the *x86* platform, or `dotnet build` / `dotnet run` from the project folder.
7. **Log in** with a role, register a test student with a blank card, deposit cash and make a test payment.

## Build and deployment

- `Build-Package.ps1` runs `dotnet publish` for `win-x86`, self-contained, and packages the output.
- `Setup.bat` installs the packaged app to `%LOCALAPPDATA%\TransactionSystem` and creates a desktop shortcut; it warns if the reader DLLs are missing.
- Every till must get the **same** `appsecrets.json` (in particular the same `CardMasterKey`), or cards written at one till will fail signature checks at another.

## Conventions

- Money operations: card write first, confirmed by read-back, then one MySQL transaction; never the other way round.
- Every money operation carries a GUID `TransactionID` generated before the card write.
- Secrets live only in `appsecrets.json`; nothing secret is committed.
- Branch per feature; commit messages describe the change and the use case it touches.